

- *Having two ears*, one on the left and one on the right, gives us a great deal of positional information. A sound that is to our right will sound louder in the right ear than in the left. An *interaural time difference* (ITD) of approximately one millisecond also arises, because a sound to one side of your head will take just a little bit longer to reach the opposite ear. Finally, the head itself obstructs sounds, so the ear opposite to the sound source will perceive a slightly muffled version of the sound reaching the near ear. This is known as interaural intensity difference (IID).
- *Ear shape* has an effect as well. Our ears are cupped slightly forward, so sounds coming from behind us are very slightly muffled relative to those coming from in front of us.
- *The head-related transfer function* (HRTF) is a mathematical model of the minute effects that the folds of our ears (the *pinnae*) have on sounds coming from different directions.

14.2 The Mathematics of Sound

Signal processing and systems theory is the area of mathematics that lies at the heart of virtually all modern audio technology. It is also extensively used in a wide variety of other technological and engineering endeavors, including image processing and machine vision, aeronautics, electronics, fluid dynamics, and the list goes on. In this section, we'll embark on a whirlwind tour of the key concepts in signals and systems theory, because it will help us to understand some of the more advanced topics in game audio later in the chapter. (It's also an important area of mathematical theory that can benefit any game programmer—so what the heck!) An in-depth treatment of the topic can be obtained from [41].

14.2.1 Signals

A *signal* is any function of one or more independent variables, typically describing the behavior of some kind of physical phenomenon. In Section 14.1, we used the signal $p(t)$ to represent the time-varying acoustic pressure of an audio compression wave. Of course, many other kinds of signals are possible. A signal $v(t)$ might represent the voltage produced by a microphone over time, while $w(t)$ might model the time-varying water pressure in a system of pipes, or we could use $f(t)$ to represent the varying population of foxes in an ecosystem.

In studying signal theory, we often refer to the independent variable as “time” and represent it with the symbol t —but of course the independent vari-

able might represent some other quantity, and there may be more than one independent variable. For example, one can think of a 2D greyscale image as a signal $i(x, y)$, where the two independent variables, x and y , represent the orthogonal coordinate axes, and the signal value i represents the intensity of the greyscale image at each pixel. A color image could be similarly represented by three signals, $r(x, y)$ for the red channel, $g(x, y)$ for the green channel and $b(x, y)$ for the blue channel.

14.2.1.1 Be Discrete, Continuously

The 2D image examples above bring to light an important distinction between two fundamental kinds of signal: *continuous* and *discrete*.

- If the independent variable is a *real number* ($t \in \mathbb{R}$), we call the signal a *continuous-time signal*. In this chapter, we'll use the symbol t to represent continuous "time," and we'll use round parentheses for our functional notation (e.g., $x(t)$) to remind us that we're working with a continuous-time signal.
- If the independent variable(s) is an *integer* ($n \in \mathbb{I}$), we call the signal a *discrete-time signal*. We'll use the symbol n to represent discrete "time," and we'll use square brackets for our functional notation (e.g., $x[n]$) to remind us that we're working with a discrete-time signal. Note that the *value* of a discrete-time signal might still be a real number ($x[n] \in \mathbb{R}$)—the only thing the term "discrete-time signal" prescribes is that the *independent* variable is an integer ($n \in \mathbb{I}$).

In Figure 14.1, we saw that we can visualize a continuous-time signal as an ordinary function plot, with time t on the horizontal axis and the signal value $p(t)$ on the vertical axis. We can plot a discrete-time signal $x[n]$ in similar fashion, although the function's values are only defined for integer values of the independent variable n (see Figure 14.7). One common way to think of a discrete-time signal is as a *sampled* version of a continuous-time signal. The sampling process (also known as *digitization* or *analog-to-digital conversion*) lies at the heart of digital audio recording and playback. See Section 14.3.2.1 for more information on sampling.

14.2.2 Manipulating Signals

It will be important in the following discussions for us to understand some basic ways to manipulate a signal by making changes to its independent variable. For example, to *reflect* a signal about $t = 0$, we simply replace t with $-t$ in the signal's equation. To *time-shift* the entire signal to the *right* (i.e., in the

positive direction) by a distance s , we replace t with $t - s$ in the signal's equation. (Time shifting to the left/negative direction is accomplished by replacing t with $t + s$.) We can also expand or compress the domain of the signal by scaling the independent variable. These simple transformations are illustrated in Figure 14.8.

14.2.3 Linear Time-Invariant (LTI) Systems

In the context of signal processing theory, a *system* is defined as any device or process that transforms an *input* signal into a new *output* signal. The mathematical concept of a system can be used to describe, analyze and manipulate many real-world systems that arise in audio processing, including microphones, speakers, analog-to-digital converters, reverb units, equalizers and filters and even the acoustics of a room.

As a simple example, an *amplifier* is a system that increases the amplitude of its input signal by a factor A known as the *gain* of the amp. Given an input signal $x(t)$, such an amplification system would produce an output signal $y(t) = Ax(t)$.

A *time-invariant* system is one for which a time shift in the input signal causes an equal time shift in the output signal. In other words, the behavior of the system does not change over time.

A *linear* system is one that possesses the property of *superposition*. This means that if an input signal consists of a *weighted sum* of other signals, then the output is a weighted sum of the individual outputs that would have been produced, had each of the other signals been fed through the system independently.

Linear time-invariant (LTI) systems are extremely useful for two reasons.

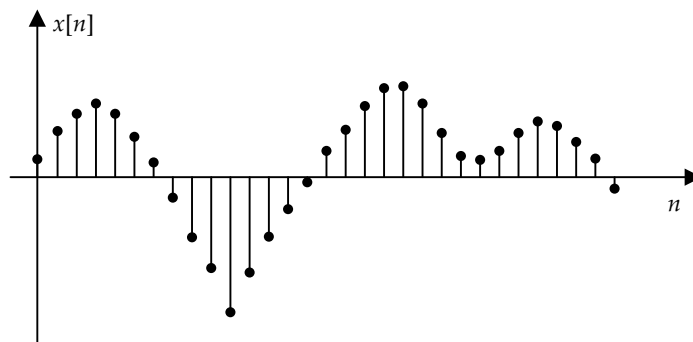


Figure 14.7. The value of a discrete-time signal $x[n]$ is defined only for integer values of n .

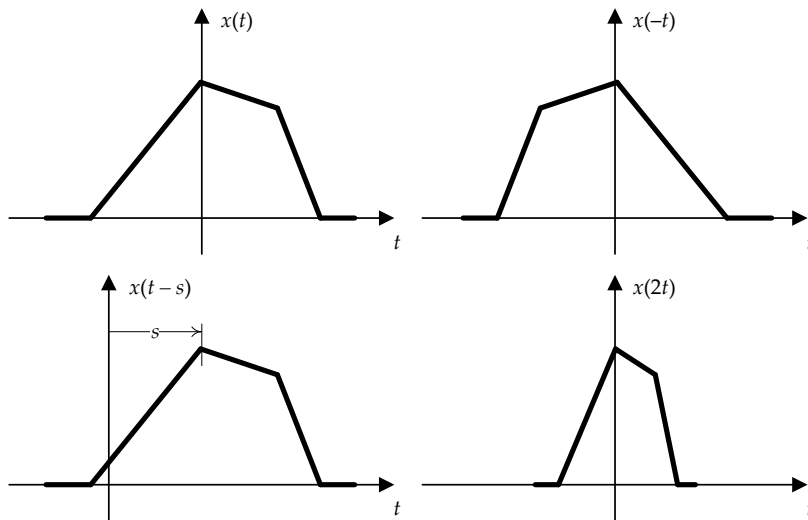


Figure 14.8. Simple manipulations of a signal's independent variable.

First, their behaviors are well understood and relatively easy to work with mathematically. Second, many real physical systems in the fields of audio propagation theory, electronics, mechanics, fluid flow, etc. can be modeled accurately using LTI systems. As such, we will restrict ourselves to a discussion of LTI systems for our purposes of understanding audio technology.

We can visualize any system as a black box with an input signal and an output signal, as shown in Figure 14.9.

Using this black-box notation, simple systems can be conveniently interconnected to construct more complex systems. For example:

- The output of system A could be connected to the input of system B, yielding a composite system that performs operation A followed by operation B. This is called a *serial connection*.
- The outputs of two systems could be added together.
- The output of a system could be *fed back* into an earlier input, yielding what is known as a *feedback loop*.

See Figure 14.10 for examples of all of these kinds of connections.

One very important property of all LTI systems is that their interconnections are *order-independent*. So if we have a serial connection of system A followed by system B, we can reverse the order of the two systems and the output will remain unchanged.

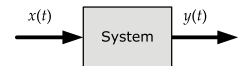


Figure 14.9. A system as a black box.

14.2.4 Impulse Response of an LTI System

It's all fine and dandy to talk about systems that convert an input signal into an output signal, and it's even pretty intuitive to draw diagrams of system interconnections. But how can we describe the operation of a system mathematically?

Recall from Section 14.2.3 that for a *linear* system, if the input consists of a linear combination (weighted sum) of input signals, the output will be a linear combination (weighted sum) of the individual outputs (had each of the input signals been fed into the system independently). So, if we can figure out a way to represent an arbitrary input signal as a weighted sum of very simple signals, we should be able to describe the behavior of the system by describing *only* its response to those very simple signals.

14.2.4.1 The Unit Impulse

If we are going to describe an input signal as a linear combination of simple signals, the question arises: Which simple signal shall we use? For reasons that will become clear in a moment, our signal of choice is going to be the *unit impulse*. This signal is one of a family of related functions known as *singularity functions* because they all contain at least one discontinuity or “singularity.”

In discrete time, the unit impulse $\delta[n]$ is as simple as it gets: It is a signal whose value is zero everywhere except at $n = 0$, where its value is one:

$$\delta[n] = \begin{cases} 1 & \text{if } n = 0, \\ 0 & \text{otherwise.} \end{cases}$$

The discrete-time unit impulse is illustrated in Figure 14.11.

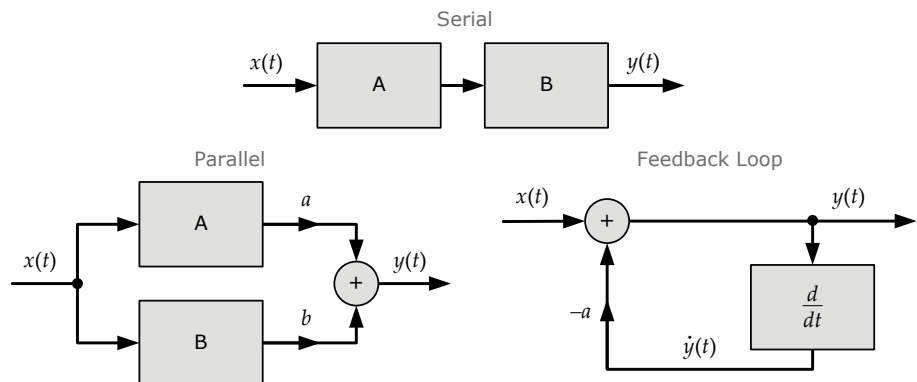


Figure 14.10. Various ways to interconnect systems. In the serial connection, $y(t) = B(A(x(t)))$. In the parallel connection, $y(t) = aA(x(t)) + bB(x(t))$. In the feedback loop, $y(t) = x(t) - a\dot{y}(t)$.

In continuous time, the unit impulse $\delta(t)$ is a bit trickier to define. It is a function whose value is zero everywhere except at $t = 0$, where its value is infinite—but the *area* under the curve is equal to one.

To see how such a strange beast of a function might be formally defined, imagine a “box” function $b(t)$, whose value is zero everywhere except in the interval $[0, T)$, where its value is $1/T$. The area under this curve is just the area of the box, width times height, or $T \times \frac{1}{T} = 1$. Now imagine the limit as $T \rightarrow 0$. As this happens, the width of the box approaches zero and its height approaches infinity, but its area remains equal to 1. This is shown in Figure 14.12.

The unit impulse function is typically denoted by the symbol $\delta(t)$. It can be formally defined as follows:

$$\delta(t) = \lim_{T \rightarrow 0} b(t),$$

where

$$b(t) = \begin{cases} 1/T & \text{if } t \geq 0 \text{ and } t < T, \\ 0 & \text{otherwise.} \end{cases}$$

As shown in Figure 14.13, we typically plot the unit impulse by drawing an arrow whose height represents the area under the curve (since the actual “height” of the function at $t = 0$ is infinite).

14.2.4.2 Using an Impulse Train to Represent a Signal

Now that we know what the unit impulse signal is, let’s see if we can describe an arbitrary signal $x[n]$ as a linear combination of unit impulses. (Spoiler alert: It turns out we can.)

The function $\delta[n - k]$ is a time-shifted discrete unit impulse, whose value is zero everywhere except at time $n = k$, where it is equal to one. In other words, the unit impulse $\delta[n - k]$ is “positioned” at time k . Consider an impulse at one particular value of k (say, $k = 3$). Let’s make sure that the “height” of that impulse matches the value of the original function at $k = 3$ by “scaling” the impulse by $x[3]$, yielding $x[3]\delta[n - 3]$. If we rinse and repeat for all possible

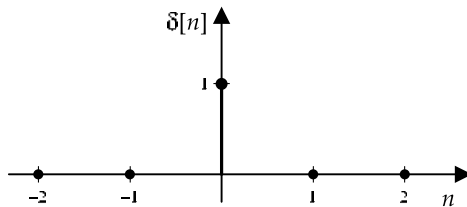


Figure 14.11. The unit impulse in discrete time.

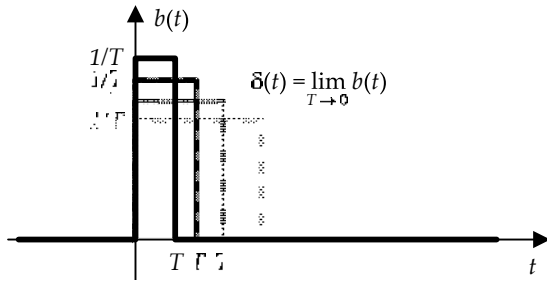


Figure 14.12. The unit impulse can be defined as the limit of a box function $b(t)$ whose width approaches zero.

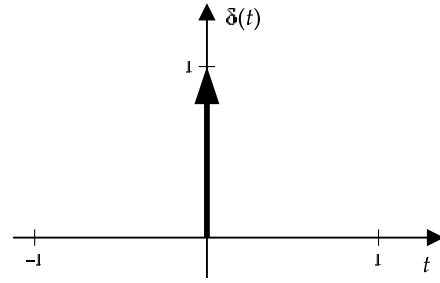


Figure 14.13. The value of the unit impulse function $\delta(t)$ is zero everywhere except at $t = 0$, where it is infinite. It is drawn as an arrow of unit height to indicate that the area under the curve is 1.

values of k , we get a train of impulses of the form $x[k]\delta[n - k]$. Adding all these scaled, time-shifted impulse functions together is just another way of writing the original signal $x[n]$:

$$x[n] = \sum_{k=-\infty}^{+\infty} x[k]\delta[n - k]. \quad (14.4)$$

We won't give a rigorous proof here, but it's probably not too hard to believe that doing this in continuous time works in pretty much the same way. The only difficulty is that for continuous time, the sum in Equation (14.4) becomes an integral. Let's imagine an infinite sequence of time-shifted unit impulses $\delta(t - \tau)$, each one located at a different time τ . We can build up an arbitrary signal $x(t)$ in an analogous fashion to the discrete-time case, as follows:

$$x(t) = \int_{\tau=-\infty}^{+\infty} x(\tau)\delta(t - \tau) d\tau. \quad (14.5)$$

14.2.4.3 Convolution

Equation (14.4) tells us how to represent a signal $x[n]$ as a *linear combination* of simple, time-shifted unit impulse signals $\delta[n - k]$. Let's imagine putting just *one* of these weighted impulse inputs ($x[k]\delta[n - k]$) through the system. It doesn't matter which one we choose, so let's select the one at $k = 0$. This gives us the input signal $x[0]\delta[n]$.

We'll use the notation $x[n] \Rightarrow y[n]$ to indicate that an input signal $x[n]$ is being transformed by an LTI system into an output signal $y[n]$. So we can write:

$$x[0]\delta[n] \Rightarrow y[n].$$

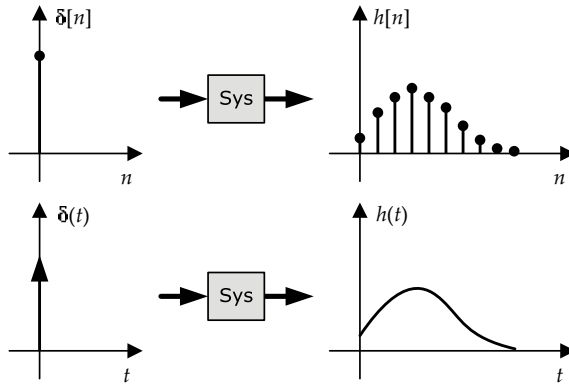


Figure 14.14. Examples of the impulse response of a system in discrete and continuous time.

The value of $x[0]$ is just a constant, so because we're dealing with a linear system, the output $y[n]$ will just be that same constant times the system's response to the unit impulse $\delta[n]$. Let's use the signal $h[n]$ to represent the system's response to a "bare" unit impulse: $\delta[n] \Rightarrow h[n]$. The signal $h[n]$ is called the *impulse response* of the system. So we can write the system's response to our simple input signal as follows:

$$x[0]\delta[n] \Rightarrow x[0]h[n].$$

The concept of impulse response is illustrated in Figure 14.14.

The response of an LTI system to a time-shifted unit impulse is just a time-shifted impulse response ($\delta[n - k] \Rightarrow h[n - k]$). So for values of k other than zero, everything works out exactly the same except that now the input and output signals are both *time-shifted* by k :

$$x[k]\delta[n - k] \Rightarrow x[k]h[n - k].$$

To find the system's response to the entire input signal $x[n]$, we just sum up the responses to each individual time-shifted component, like this:

$$\sum_{k=-\infty}^{+\infty} x[k] \delta[n - k] \Rightarrow \sum_{k=-\infty}^{+\infty} x[k] h[n - k].$$

In other words, the output of our system can be written as follows:

$$y[n] = \sum_{k=-\infty}^{+\infty} x[k] h[n - k]. \quad (14.6)$$

This very important equation is known as the *convolution sum*. It's convenient to introduce a new mathematical operator $*$ to represent the operation

of convolution:

$$x[n] * h[n] = \sum_{k=-\infty}^{+\infty} x[k] h[n - k]. \quad (14.7)$$

Equations (14.6) and (14.7) give us a way to calculate an LTI system's response $y[n]$ to any arbitrary input signal $x[n]$, given only the *impulse response* of the system, $h[n]$. In other words, for LTI systems, the impulse response signal $h[n]$ *completely describes the system*. Pretty cool stuff.

Convolution in Continuous Time

In our discussions above, we worked in discrete time to keep things simple. In continuous time, everything works out in pretty much the same way. The only difference is that summations become integrals, and we need to remember to include the differential $d\tau$ in our equations.

When we apply an arbitrary signal $x(t)$ to the input of a continuous-time LTI system, the output signal can be written as follows:

$$y(t) = \int_{\tau=-\infty}^{+\infty} x(\tau) h(t - \tau) d\tau. \quad (14.8)$$

As before, we'll use the operator $*$ as a shorthand for convolution:

$$x(t) * h(t) = \int_{\tau=-\infty}^{+\infty} x(\tau) h(t - \tau) d\tau. \quad (14.9)$$

Analogous to the convolution sum, the integral in Equations (14.8) and (14.9) is known as the *convolution integral*.

14.2.4.4 Visualizing Convolution

Let's try to visualize the convolution operation in the continuous time case. To evaluate $y(t) = x(t) * h(t)$ for one specific value of t (say, $t = 4$), we perform the following steps as illustrated in Figure 14.15:

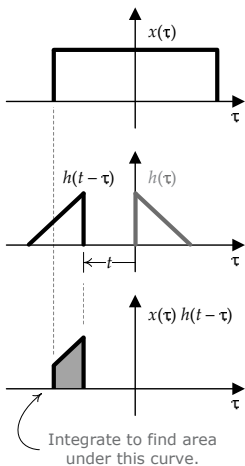


Figure 14.15. Visualization of the convolution operation in continuous time.

1. Plot $x(\tau)$, using τ as the time variable because t is fixed (at $t = 4$ in this example).
2. Plot $h(t - \tau)$. We can rewrite this as $h(-\tau + t)$. Because τ is negated, we know that the impulse response has been flipped about $\tau = 0$. And because we've added t to the independent variable, we know the signal has been shifted to the *left* by $t = 4$ units.
3. Multiply the two signals together across the entire τ axis.

4. Integrate from $-\infty$ to $+\infty$ along the τ axis to find the area under the resulting curve. This is the value of $y(t)$ at this one specific value of t (in this example, $t = 4$).

Remember that we must repeat this procedure for every possible value of t in order to determine the complete output signal $y(t)$.

14.2.4.5 Some Properties of Convolution

The properties of the convolution operation are surprisingly analogous to those of ordinary multiplication. Convolution is:

- *commutative*: $x(t) * h(t) = h(t) * x(t)$;
- *associative*: $x(t) * (h_1(t) * h_2(t)) = (x(t) * h_1(t)) * h_2(t)$; and
- *distributive*: $x(t) * (h_1(t) + h_2(t)) = (x(t) * h_1(t)) + (x(t) * h_2(t))$.

14.2.5 The Frequency Domain and the Fourier Transform

In order to arrive at the concepts of impulse response and convolution, we described a signal as a weighted sum of unit impulses. We can also represent a signal as a weighted sum of sinusoids. Representing a signal in this manner essentially breaks it up into its *frequency components*. This will allow us to derive another incredibly powerful mathematical tool—the *Fourier transform*.

14.2.5.1 The Sinusoidal Signal

A sinusoidal signal is produced when a circular motion in two dimensions is projected onto a single axis. An audio signal in the form of a sinusoid produces a “pure” tone at one specific frequency.

The most basic sinusoidal signal is the sine (or cosine) function. The signal $x(t) = \sin t$ takes on the value 0 at $t = 0, \pi$ and 2π , has a value of 1 at $t = \frac{\pi}{2}$ and has a value of -1 at $t = \frac{3\pi}{2}$.

The most general form of a real-valued sinusoidal signal is

$$x(t) = A \cos(\omega_0 t + \phi). \quad (14.10)$$

Here, A represents the *amplitude* of the sine wave (i.e., the peaks and troughs of the cosine wave hit maximum and minimum values of A and $-A$, respectively). The *angular frequency* is ω_0 , measured in radians/second (see Section 14.1.1 for a discussion of frequency and angular frequency). ϕ represents a *phase offset* (also measured in radians) that shifts the cosine wave to the left or right along the time axis.

When $A = 1$, $\omega_0 = 1$ and $\phi = 0$, Equation (14.10) reduces to $x(t) = \cos t$. When $\phi = \frac{\pi}{2}$, the expression becomes $x(t) = \sin t$. The cos function represents the projection of a circular motion onto the horizontal axis, while sin represents its projection onto the vertical axis.

14.2.5.2 The Complex Exponential Signal

The cosine function isn't actually the best tool for representing a signal as a sum of sinusoids. The math is much simpler and more elegant if we make use of *complex numbers* instead. In order to understand how this works, we need to review complex math, and take a look at how multiplication of complex numbers works. So bear with me here—all will become clear by the time we're done.

A Brief Review of Complex Numbers

You'll probably remember from high-school math class that a complex number is a kind of two-dimensional quantity consisting of a real part and an imaginary part. Any complex number can be written as follows: $c = a + jb$, where a and b are real numbers and $j = \sqrt{-1}$ is the *imaginary unit*. The real part of c is $a = \Re(c)$, and its imaginary part is $b = \Im(c)$.

You can visualize a complex number as a kind of “vector” $[a, b]$ in a two-dimensional space known as the *Argand plane*. It's important to remember, however, that complex numbers and vectors are *not* interchangeable—their mathematical behaviors are quite different.

We define the *magnitude* of a complex number as the length of its 2D “vector” representation in the complex plane: $|c| = \sqrt{a^2 + b^2}$. The angle the vector makes with the real axis is known as its *argument*: $\arg c = \tan^{-1}(b/a)$. (The argument of a complex number is sometimes called its *phase*. As we'll see, the term “phase” is closely related to the *phase offset* ϕ in Equation (14.10).) The magnitude and argument of a complex number are depicted in Figure 14.16.

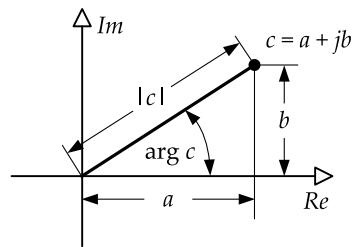


Figure 14.16. The magnitude $|c| = \sqrt{a^2 + b^2}$ of a complex number is its length in the complex plane, and its argument $\arg c = \tan^{-1}(b/a)$ is the angle it makes with the $\Re$ axis.

Complex Multiplication and Rotation

We won't get into all of the properties of complex numbers here. Check out <http://www.math.wisc.edu/~angenent/Free-Lecture-Notes/freecomplexnumbers.pdf> for an in-depth discussion of complex number theory. However, there is one mathematical operation that does concern us here: the operation of complex *multiplication*.

Complex numbers are multiplied algebraically (no dot or cross products here):

$$\begin{aligned} c_1 c_2 &= (a_1 + j b_1)(a_2 + j b_2) \\ &= (a_1 a_2) + j(a_1 b_2 + a_2 b_1) + j^2 b_1 b_2 \\ &= (a_1 a_2 - b_1 b_2) + j(a_1 b_2 + a_2 b_1). \end{aligned} \quad (14.11)$$

If you work out² the magnitude and argument (angle) of the product $c_1 c_2$, you'll find that the magnitude is equal to the *product* of the two input magnitudes, and the argument is the *sum* of the input arguments:

$$\begin{aligned} |c_1 c_2| &= |c_1| |c_2|; \\ \arg(c_1 c_2) &= \arg c_1 + \arg c_2. \end{aligned} \quad (14.12)$$

The fact that multiplication of complex numbers causes their angles (arguments) to *add* means that complex multiplication produces a *rotation* in the complex plane. If the magnitude of c_1 is unity ($|c_1| = 1$), then the magnitude of the product will be equal to the magnitude of c_2 ($|c_1 c_2| = |c_2|$). In this case, the product represents a *pure rotation* of c_2 by an angle equal to $\arg c_1$ (see Figure 14.17). If $|c_1| \neq 1$, then the product's magnitude will be *scaled* by $|c_1|$, and the result is that c_2 undergoes a *spiral motion* in the complex plane.

This explains why *unit-length* quaternions operate as rotations in 3D space! A quaternion is essentially a four-dimensional complex number, with one real part and three imaginary parts. So a quaternion follows the same basic rules in three dimensions that a regular complex number follows in two dimensions.

The fact that complex multiplication produces a rotation makes sense when we consider what happens when we multiply j by itself many times:

$$\begin{aligned} 1 \times j &= j, \\ j \times j &= \sqrt{-1} \sqrt{-1} = -1, \\ -1 \times j &= -j, \\ -j \times j &= 1, \\ &\dots \end{aligned}$$

²Yikes—this sounds an awful lot like an exercise for the reader...

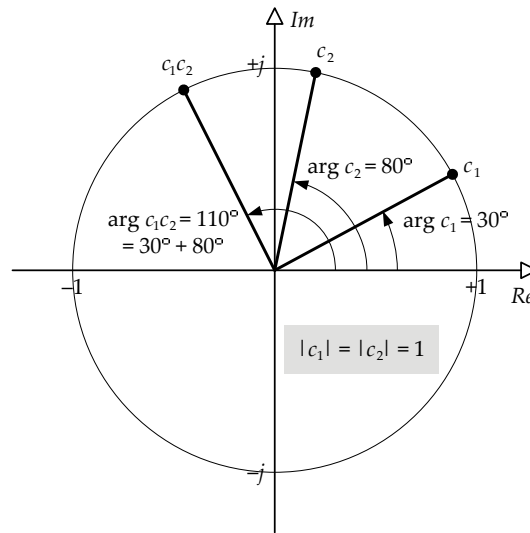


Figure 14.17. Multiplying two complex numbers together that both have magnitudes of 1 produces a *pure rotation* in the complex plane.

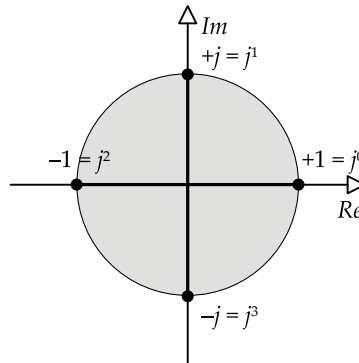


Figure 14.18. Multiplying the imaginary number $j = \sqrt{-1}$ by itself acts like rotating a unit vector by 90 degrees in the complex plane.

So multiplying j by itself is like *rotating* a unit vector by 90 degrees in the complex plane. In fact, multiplying *any* complex number by j has the effect of rotating it by 90 degrees. This is illustrated in Figure 14.18.

The Complex Exponential and Euler's Formula

For any complex number c with $|c| = 1$, the function $f(n) = c^n$, with n taking on a sequence of increasing positive real values, will trace out a *circular path* in

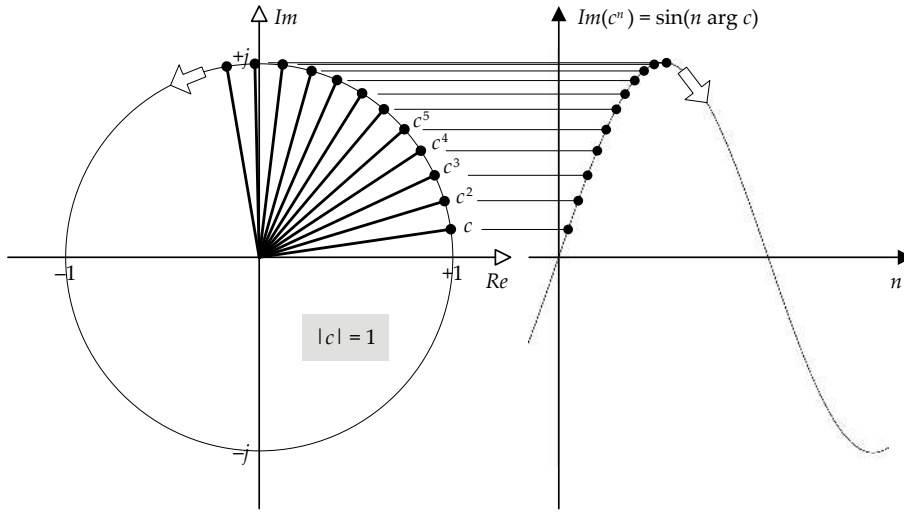


Figure 14.19. Multiplying a complex number by itself repeatedly traces out a circular path in the Argand plane, producing a sinusoid when projected onto any axis through the origin.

the complex plane. Any circular path in two dimensions traces out a sine curve along the vertical axis, and a corresponding cosine curve along the horizontal axis. This is illustrated in Figure 14.19.

Raising a *complex* number to a *real* power (c^n) produces rotation in the complex plane, and therefore yields a sinusoid when projected onto any axis in the plane. As it turns out, we can also get this rotational effect by raising a *real* number to a *complex* power (n^c). This means that we can write Equation (14.10) in terms of complex numbers as follows:

$$\begin{aligned}
 e^{j\omega_0 t} &= \cos \omega_0 t + j \sin \omega_0 t, \quad t \in \mathbb{R}; \\
 \Re[e^{j\omega_0 t}] &= \cos \omega_0 t; \\
 \Im[e^{j\omega_0 t}] &= \sin \omega_0 t,
 \end{aligned} \tag{14.13}$$

where $e \approx 2.71828$ is the real transcendental number that defines the base of the natural logarithm function.

Equation (14.13) is one of the most important equations in all of mathematics. It is known as *Euler's formula*. Why it works is a bit of a mystery (even to some seasoned mathematicians). The theorem can be explained by looking at the Taylor series expansion of e^{jt} , or by considering the derivative of e^x and then allowing x to become a complex number. But for our purposes, it should suffice to rely on the intuitions we gained from looking at how complex multiplication results in rotation in the complex plane.

14.2.5.3 The Fourier Series

Now that we have the mathematical tools we need to represent sinusoids as complex numbers, let's turn our attention again to the task of representing a signal as a sum of sinusoids.

Doing this is easiest when the signal is *periodic*. In this case, we can write the signal as a sum of *harmonically related* sinusoids:

$$x(t) = \sum_{k=-\infty}^{+\infty} a_k e^{j(k\omega_0)t}. \quad (14.14)$$

We call this the *Fourier series* representation of the signal. Here, the complex exponential functions $e^{j(k\omega_0)t}$ are the sinusoidal components from which we are building up the signal. These components are *harmonically related*, in that each one has a frequency that is an integer multiple k of the so-called *fundamental frequency* ω_0 . The coefficients a_k represent the “amount” of each harmonic present in the signal $x(t)$.

14.2.5.4 The Fourier Transform

A full explanation of this topic is beyond the scope of this book, but for our purposes it will suffice to state (without any proof whatsoever!) that *any* reasonably well-behaved signal,³ even signals that are *non-periodic*, can be represented as a linear combination of sinusoids. In general, an arbitrary signal may contain components at *any* frequency, not just frequencies that are harmonically related. As such, the discrete set of harmonic coefficients a_k in Equation (14.14) becomes a continuum of values representing “how much” of each frequency the signal contains.

We can envision a new function $X(\omega)$ whose independent variable is the frequency ω rather than time t , and whose value represents the amount of each frequency present in the original signal $x(t)$. We say that $x(t)$ is the *time domain* representation of the signal, while $X(\omega)$ is its *frequency domain* representation.

Mathematically, we can find the frequency domain representation of a signal from its time domain representation, and vice versa, by using the *Fourier transform*:

$$X(\omega) = \int_{-\infty}^{+\infty} x(t) e^{-j\omega t} dt; \quad (14.15)$$

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} X(\omega) e^{j\omega t} d\omega. \quad (14.16)$$

³All signals that meet the so-called *Dirichlet conditions* have Fourier transforms and are therefore “reasonably well-behaved” for our purposes.

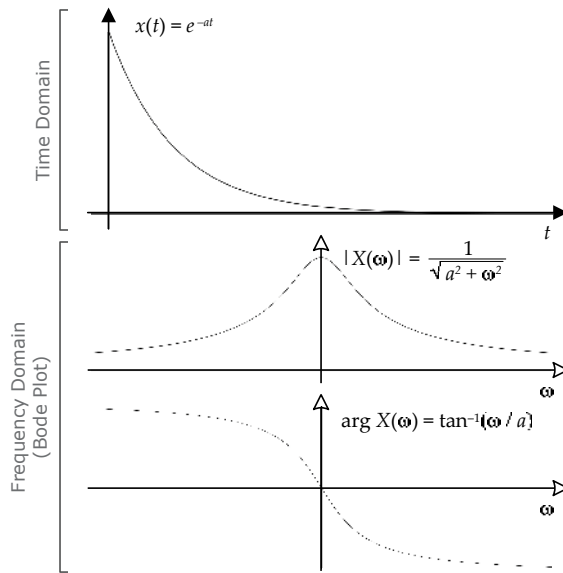


Figure 14.20. The Fourier transform yields a complex-valued frequency domain signal. A Bode plot is used to visualize this complex-valued signal in terms of its magnitude and its phase (or argument).

If you compare Equation (14.16) to the Fourier series from Equation (14.14), you can see the similarity. Rather than describing the “amounts” of the frequency components via a discrete series of coefficients a_k , we’re now describing them using the continuous function $X(\omega)$. But in both cases we’re representing $x(t)$ as a “sum” of sinusoids.

14.2.5.5 Bode Plots

In general the Fourier transform of a real-valued signal is a *complex-valued* signal ($X(\omega) \in \mathbb{C}$). When visualizing the Fourier transform, we often draw it using two plots. For example, we might plot its real and imaginary components. Or we might plot its magnitude and its argument (angle) on two different plots—a visualization known as a *Bode plot* (pronounced “Boh-dee”). Figure 14.20 shows an example of a signal and its Bode plot.

14.2.5.6 The Fast Fourier Transform (FFT)

A collection of fast algorithms exist for calculating the Fourier transform in discrete time. This family of algorithms is called, aptly enough, the *fast Fourier transform* or FFT. You can read more about the FFT at http://en.wikipedia.org/wiki/Fast_Fourier_transform.

14.2.5.7 Fourier Transforms and Convolution

It is interesting to note that convolution in the time domain corresponds to multiplication in the frequency domain and vice versa. Given a system whose impulse response is $h(t)$, we know that we can find the output of the system $y(t)$ in response to an input $x(t)$ as follows:

$$y(t) = x(t) * h(t).$$

In the frequency domain, given the Fourier transforms of the impulse response $H(\omega)$ and the input $X(\omega)$, we can find the Fourier transform of the output as follows:

$$Y(\omega) = X(\omega)H(\omega).$$

This result is pretty incredible, and it's also very handy. Sometimes it is more convenient to perform a convolution on the time axis using a system's impulse response $h(t)$, while at other times it's more convenient to perform a multiplication in the frequency domain using the system's *frequency response* $H(\omega)$.

As it turns out, LTI systems exhibit a property called *duality*, which says that you can reverse the roles of time and frequency and virtually the same mathematical rules continue to apply. So, for example, we can understand how signal *modulation* (the multiplication of one signal by another) works in the time domain by looking at what happens when we convolve the Fourier transforms of the two signals on the frequency axis. Having two ways of tackling a problem is always better than one!

14.2.5.8 Filtering

The Fourier transform allows us to visualize the set of frequencies that make up virtually any audio signal. A *filter* is an LTI system that attenuates a selected range of input frequencies while leaving all other frequencies unaltered. A *low-pass* filter retains low frequencies while attenuating high frequencies. A *high-pass* filter does the opposite, retaining high frequencies and attenuating lower frequencies. A *band-pass* filter attenuates both low and high frequencies but retains frequencies within a limited *passband*. A *notch* filter does the opposite, retaining low and high frequencies but attenuating frequencies within a limited *stopband*.

Filters are used in a stereo system's *equalizer*, by attenuating or boosting specific frequencies based on user inputs. Filters can also be used to attenuate noise, if the spectra of the noise signal and the desirable signal occupy different regions of the frequency axis. For example, if a high-frequency noise signal is

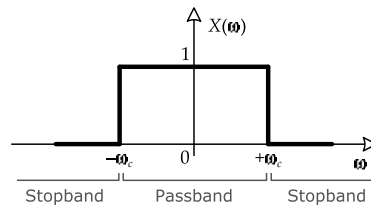


Figure 14.21. The frequency response $H(\omega)$ for an ideal filter has a value of one in the passband and zero in the stopband.

adversely affecting a lower-frequency voice or music signal, a low-pass filter could be used to eliminate the noise.

The frequency response $H(\omega)$ of an ideal filter looks like a rectangular box, with a value of one in the passband and zero in the stopband. When we multiply this by the Fourier transform of our input signal $X(\omega)$, the output $Y(\omega) = X(\omega)H(\omega)$ will have its passband frequencies preserved exactly, and its stopband frequencies all set to zero. The frequency response for an ideal filter is shown in Figure 14.21.

Of course, an ideal filter that completely passes certain frequencies and completely suppresses others may not be desirable. The frequency responses of most real-world filters have a gradual fall-off between the passband and stopband. This aids filtering in situations where there is no single, clear-cut line between the desirable frequencies and the unwanted frequencies. The frequency response of a low-pass filter with a gradual fall-off is shown in Figure 14.22.

The *equalizer* (EQ) found on most high-fidelity audio equipment permits the user to adjust the amount of bass, mid-range and treble that is output. An EQ is really just a collection of filters tuned to different frequency ranges and applied in series to an audio signal.

Filtering theory is an immense field of study, so we can't possibly do it justice here. For a great deal more information, see [41, Chapter 6].

14.3 The Technology of Sound

Before we can fully understand the software that comprises a game's audio engine, we need a firm grasp of audio hardware and technology and of the terminology used by industry professionals to describe it.